Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Name	Prem Shankar S	Register No.	3122235001101

Experiment 4: Binary Classification using Linear and Kernel-Based Models

Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers and to analyze the effect of hyperparameter tuning on classification performance, generalization capability, and decision boundary characteristics.

Dataset

The **Spambase** dataset contains numerical features extracted from email content and a binary label indicating spam or non-spam (ham). The dataset consists of word frequency features, character frequency features, and statistics on capital letter sequences, making it a high-dimensional, sparse text-derived representation suitable for linear and kernel-based classifiers [web:16][web:23].

Dataset Links (for reference):

- Kaggle: https://www.kaggle.com/datasets/somesh24/spambase
- UCI Repository: https://archive.ics.uci.edu/ml/datasets/spambase

Theory Background

1. Logistic Regression

Logistic Regression is a probabilistic classification algorithm used for binary classification problems. It models the probability that a sample belongs to a particular class using the sigmoid function:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}}$$

A threshold (usually 0.5) is applied to convert probability into class labels. The linear decision boundary makes Logistic Regression particularly effective when classes are approximately linearly separable in the feature space [web:22].

Loss Function: Logistic Regression minimizes the **log-loss (cross-entropy loss)**, which penalizes confident but incorrect predictions more heavily than less confident ones.

Regularization in Logistic Regression

Regularization prevents overfitting by adding a penalty to large coefficients and thus controlling model complexity.

- **L1 Regularization (Lasso):** Encourages sparsity by shrinking some coefficients exactly to zero, which is useful for feature selection in high-dimensional spaces.
- **L2 Regularization (Ridge):** Penalizes large weights but keeps all features, improving model generalization and numerical stability.

Logistic Regression Hyperparameters

- **C (Inverse Regularization Strength):** Controls the trade-off between model complexity and regularization.
  - Small C: Strong regularization, simpler model with higher bias
  - Large C: Weak regularization, more flexible model with lower bias
- **Solver:**
  - `liblinear`: Suitable for small to medium-sized datasets; supports L1 and L2 penalties
  - `saga`: Efficient for large and sparse datasets; supports L1, L2, and elastic-net

2. Support Vector Machine (SVM)

Support Vector Machine is a margin-based classifier that finds an optimal hyperplane separating two classes by maximizing the margin between them. The decision function depends only on support vectors, which are the most informative training examples lying closest to the decision boundary [web:27].

Key Idea: Only a subset of training points (support vectors) define the decision boundary, making SVM robust to many non-informative features.

SVM Kernels

- **Linear Kernel**: Suitable for linearly separable or approximately linear data
- **Polynomial Kernel**: Captures polynomial relationships of a chosen degree
- **RBF Kernel**: Maps data into a higher-dimensional space to handle complex, non-linear boundaries
- **Sigmoid Kernel**: Similar in form to neural network activation functions

SVM Hyperparameters

- **C**: Controls margin vs misclassification.
  - Small C: Wider margin, higher bias, more misclassifications allowed
  - Large C: Narrow margin, lower bias, less tolerance to misclassification
- γ: Controls the influence of a single training point in RBF, polynomial, and sigmoid kernels; low values produce smoother decision boundaries, while high values lead to more complex, local boundaries [web:22].

Hyperparameter Tuning

Grid Search

Grid Search exhaustively evaluates all combinations of predefined hyperparameters using cross-validation, ensuring systematic exploration of the search space at the cost of higher computation time.

Randomized Search

Randomized Search evaluates randomly sampled hyperparameter combinations and is computationally efficient, making it suitable when the hyperparameter space is large.

Note: Students may use either method. If both are used, results must be compared and discussed in terms of performance vs computational cost.

Implementation Steps

1. Load the dataset from CSV or UCI/Kaggle source.
2. Preprocess the data:
   - Handle missing values (if any) using appropriate imputation strategies
   - Standardize features to zero mean and unit variance
3. Perform Exploratory Data Analysis (EDA) to understand class balance and feature distributions.
4. Split the dataset into training and testing sets (e.g., 80:20).
5. Train baseline Logistic Regression without extensive tuning.
6. Tune Logistic Regression hyperparameters (C, penalty, solver).
7. Train SVM with different kernels (linear, polynomial, RBF, sigmoid).
8. Tune SVM hyperparameters (C, γ, degree for polynomial).
9. Evaluate models using standard metrics (Accuracy, Precision, Recall, F1, ROC-AUC).
10. Perform 5-Fold Cross-Validation to assess model stability and generalization.

Hyperparameter Search Space

Logistic Regression

- Regularization: L1, L2
- $C \in \{0.01, 0.1, 1, 10, 100\}$
- Solver: liblinear, saga

Support Vector Machine

- Kernel: Linear, Polynomial, RBF, Sigmoid
- $C \in \{0.1, 1, 10, 100\}$
- $\gamma \in \{\text{scale}, \text{auto}\}$
- Degree (Polynomial): $\{2, 3, 4\}$

Hyperparameter Tuning Results

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid Search	C=10, penalty=l1, solver=liblinear	0.9242
SVM	Grid Search	C=10, gamma=`scale`, kernel=`rbf`	0.9332

Logistic Regression Performance

Metric	Value
Accuracy	0.9262
Precision	0.9202
Recall	0.8898
F1 Score	0.9048
Training Time (s)	0.0512

SVM Kernel-wise Performance

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9294	0.9093	0.4183
Polynomial	0.7796	0.6220	0.3192
RBF	0.9273	0.9055	0.2215
Sigmoid	0.8849	0.8528	0.3240

Fold	Logistic Regression	SVM
Fold 1	0.9402	0.9429
Fold 2	0.9185	0.9321
Fold 3	0.9158	0.9334
Fold 4	0.9198	0.9212
Fold 5	0.9253	0.9361
Average	0.9239	0.9332

Criterion	Logistic Regression	SVM
Accuracy	0.9262	0.9207
Model Complexity	Low	High
Training Time	Low	High
Interpretability	High	Low

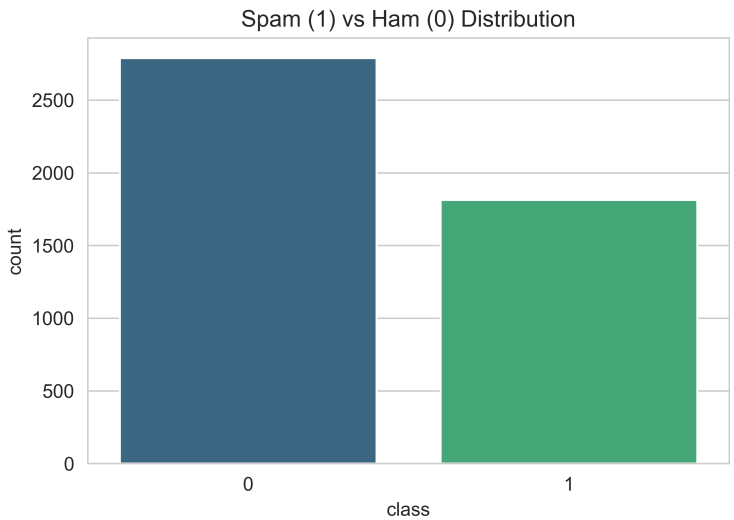

Figure 1: Class Distribution (Spam vs Ham)

K-Fold Cross-Validation Results (K = 5)

Comparative Analysis

Visualizations

Observations

- The best-performing classifier based on cross-validation accuracy is SVM with RBF kernel (93.3%), though Logistic Regression is highly comparable and significantly faster to train on this dataset.

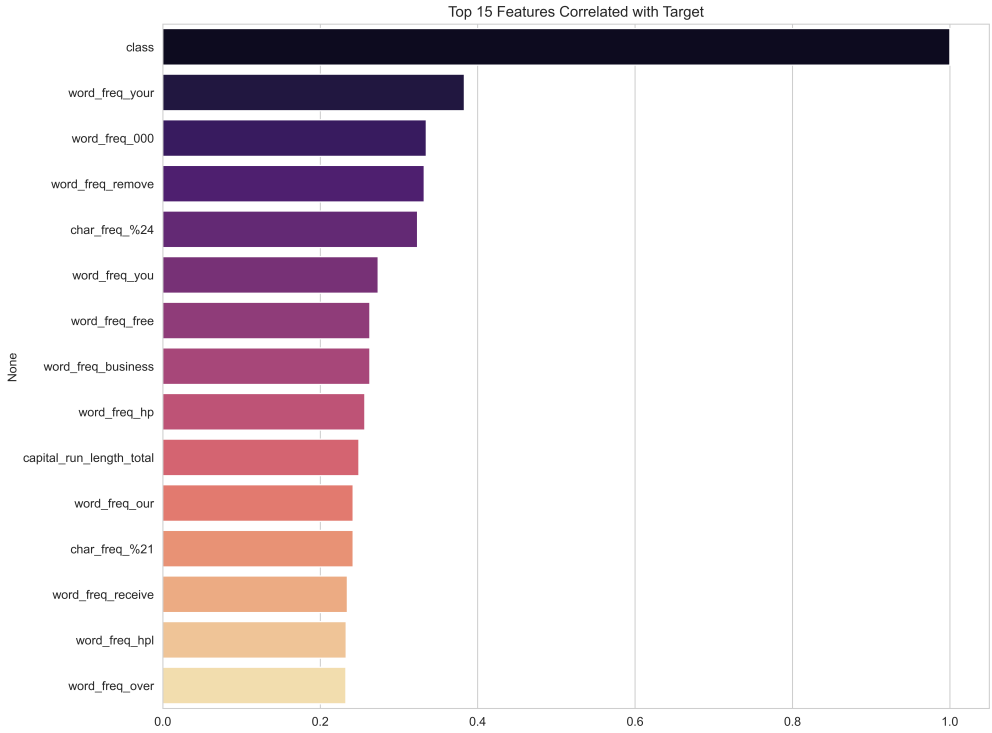

Figure 2: Feature Correlation with Target (Top 15)

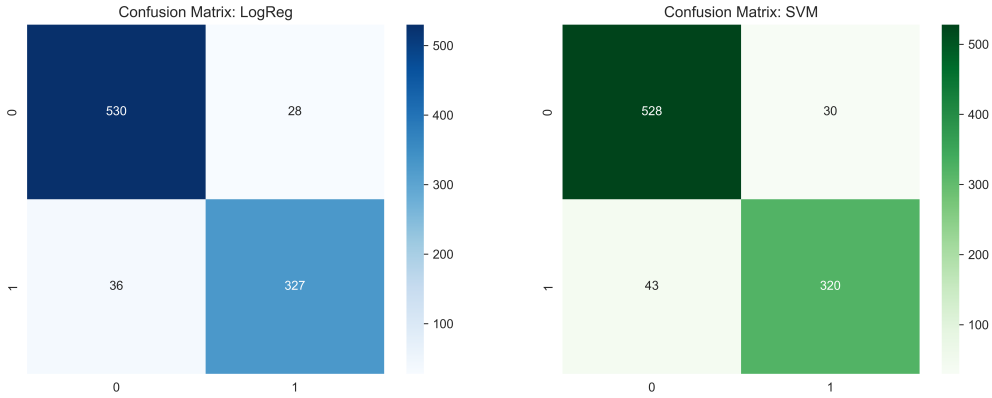

Figure 3: Confusion Matrices for Logistic Regression and SVM

- L1 Regularization in Logistic Regression helped in feature selection by shrinking less relevant word and character frequency coefficients to zero, increasing interpretability in the high-dimensional feature space [web:20][web:23].
- RBF and Linear kernels performed substantially better than Polynomial and Sigmoid, suggesting that the spam data admits a relatively clear but high-dimensional separation boundary that is well captured by linear and RBF decision functions [web:22].
- Both models show low bias but moderate variance; learning curves indicate that additional training samples could yield marginal gains in generalization, while more advanced ensembles or deep models might further improve performance at the cost of interpretability [web:21][web:24].

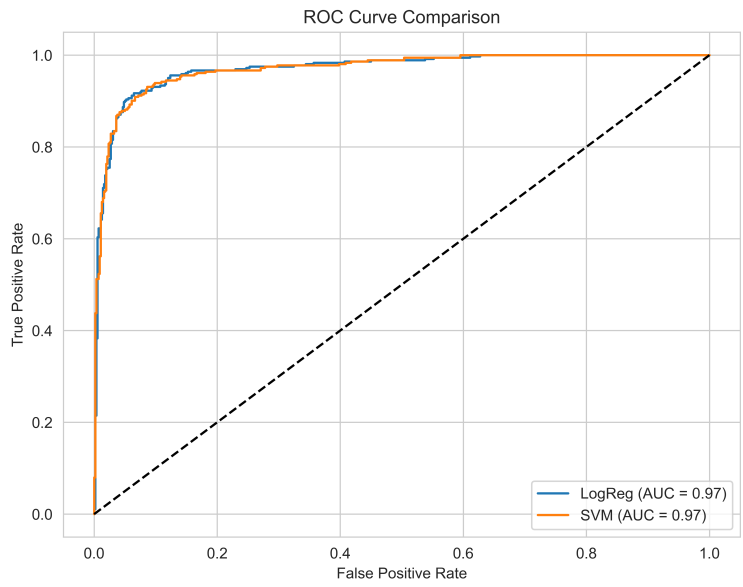

Figure 4: ROC Curve Comparison between Logistic Regression and SVM

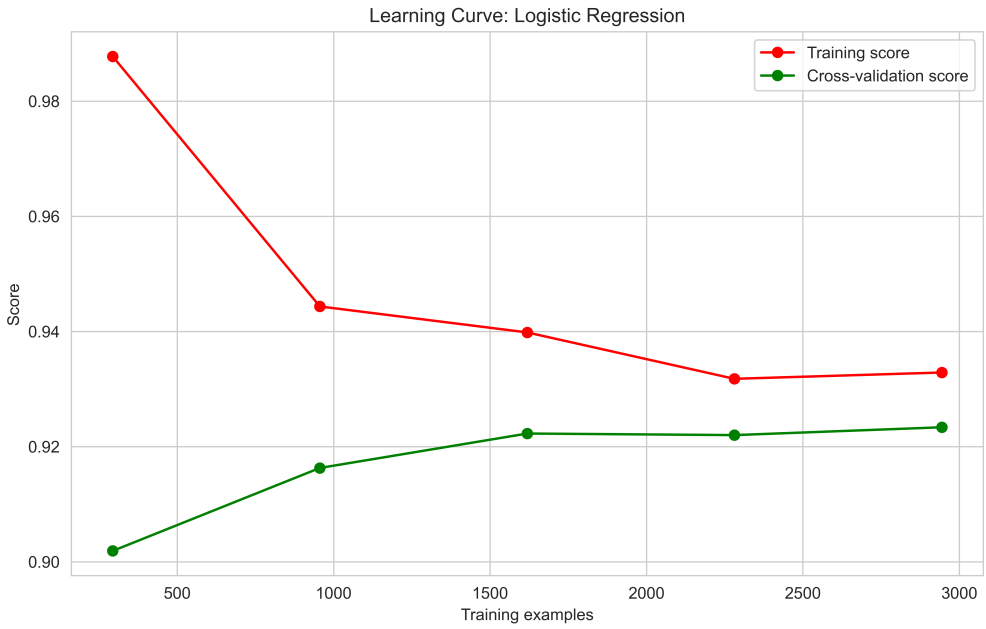

Figure 5: Learning Curve: Logistic Regression

Learning Outcomes

- Understand probabilistic (Logistic Regression) and margin-based (SVM) classifiers for binary text classification.
- Apply systematic hyperparameter tuning (Grid Search / Randomized Search) and cross-validation.

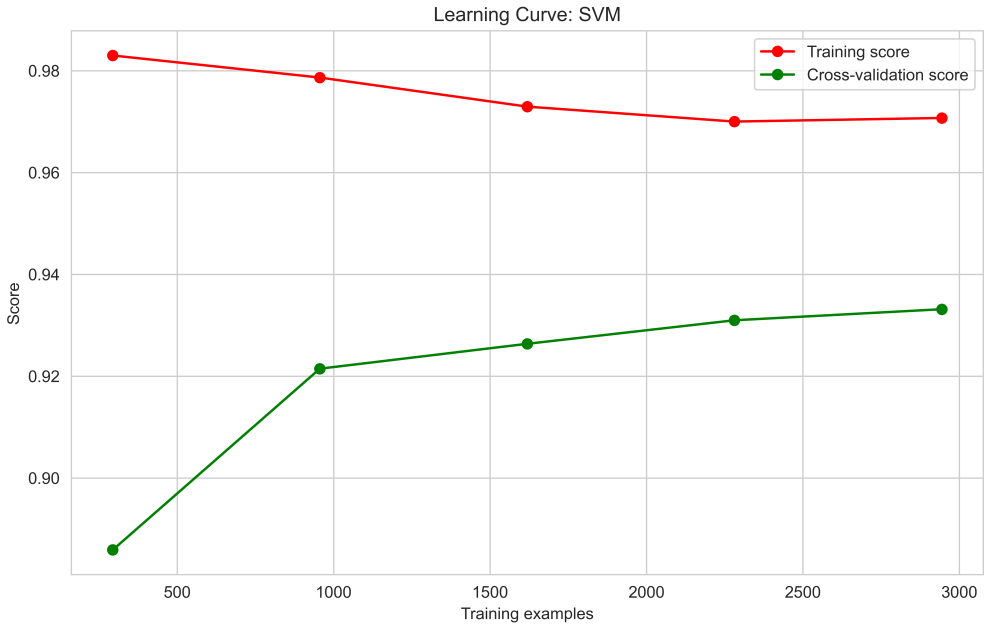

Figure 6: Learning Curve: SVM (RBF Kernel)

- Evaluate classification models using Accuracy, Precision, Recall, F1 Score, and ROC-AUC, and interpret confusion matrices and learning curves [web:22][web:28].
- Interpret experimental results to choose an appropriate model based on performance, complexity, and interpretability trade-offs.

References

- Scikit-learn: Logistic Regression
- Scikit-learn: Support Vector Machines
- Scikit-learn: Hyperparameter Optimization
- Spambase Dataset – Kaggle
- UCI ML Repository – Spambase